

Algoritmi e Strutture Dati

Analisi di algoritmi Introduzione

Alberto Montresor

Università di Trento

2026/08/21

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Sommario

- 1 Modelli di calcolo
 - Definizioni
 - Esempi
 - Ordini di complessità
- 2 Notazione asintotica
 - Definizioni
 - Esempi
- 3 Proprietà della notazione asintotica
 - Proprietà delle notazioni
 - Proprietà delle funzioni più comuni

Introduzione

Obiettivo: **stimare la complessità in tempo**

- Modelli di calcolo
- Esempi di valutazioni
- Ordini di complessità

Perché?

- Per stimare il tempo impiegato per un dato input
- Per stimare il più grande input gestibile in tempi ragionevoli
- Per confrontare l'efficienza di algoritmi diversi
- Per ottimizzare le parti più importanti

Complessità

Complessità

La **complessità** di un algoritmo è una funzione matematica **non negativa** (**funzione di costo**) che descrive l'andamento del tempo di calcolo o dello spazio utilizzato in relazione alla dimensione dell'input:

$$T : \text{"Dimensione dell'input"} \rightarrow \text{"Tempo di calcolo"}$$
$$S : \text{"Dimensione dell'input"} \rightarrow \text{"Spazio occupato"}$$

Domande

- Come definire la dimensione dell'input?
 - Come misuriamo la dimensione di un vettore?
 - Come misuriamo la dimensione di un intero?
- Come misurare il tempo?

Dimensione dell'input

Sequenze di elementi

- La dimensione naturale è il **numero di elementi** della sequenza
- Esempio: ricerca minimo in un vettore di n elementi

Valori interi

- La dimensione naturale è il **numero di bit** necessari per rappresentare l'intero
- Esempio: moltiplicazione di numeri binari lunghi n bit

In molti casi...

- Possiamo assumere che gli "elementi" siano rappresentati da un numero costante di bit
- Le due misure coincidono a meno di una costante moltiplicativa

Definizione di tempo

Tempo $\equiv$ n. istruzioni elementari

Un'istruzione si considera elementare se può essere eseguita in tempo "costante" dal processore.

Quali di queste sono operazioni elementari

- `a *= 2` ?
- `Math.cos(d)` ?
- `min(A, n)` ?

Modelli di calcolo

Modello di calcolo

Rappresentazione astratta di un calcolatore

- **Astrazione**: deve permettere di nascondere i dettagli
- **Realismo**: deve riflettere la situazione reale
- **Potenza matematica**: deve permettere di trarre conclusioni "formali" sul costo

Molte possibilità fra cui scegliere

L'articolo Wikipedia [Model of computation](#) identifica:

- tre famiglie: sequenziali, funzionali, concorrenti
- venti modelli di calcolo distinti (con sottomodelli)

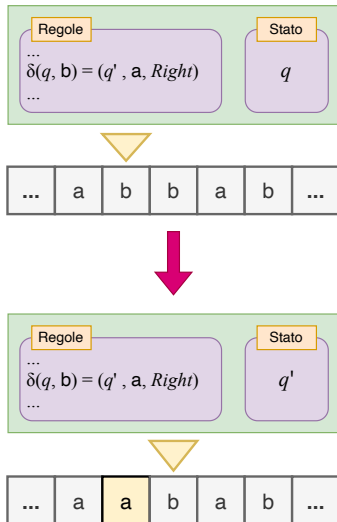
Modelli di calcolo - Macchina di Turing

Una *macchina ideale* che manipola i dati contenuti su un nastro di lunghezza infinita, secondo un insieme prefissato di regole.

Ad ogni passo, la Macchina di Turing:

- legge il simbolo sotto la testina
- modifica il proprio stato interno
- scrive un nuovo simbolo nella cella
- muove la testina a destra o a sinistra

Importante nello studio della **calcolabilità**,
ma livello troppo basso per i nostri scopi



Modelli di calcolo

Random Access Machine (RAM)

- **Memoria:**
 - Parole di w bit, indirizzate direttamente
 - Accesso in tempo costante, indipendente dall'indirizzo
- **Processore (singolo)**
 - Set di istruzioni elementari simile a quelli reali:
 - somme, sottrazioni, moltiplicazioni, operazioni logiche, etc.
 - istruzioni di controllo (salti, salti condizionati)
- **Costo delle istruzioni elementari**
 - Costante per operazioni su parole di macchina
 - Per interi più lunghi di w bit, dipende dal numero di bit

Tempo di calcolo min()

- Assumiamo $n \geq 1$ e consideriamo il caso peggiore: il vettore è strettamente decrescente
- Ogni istruzione ha un costo costante, potenzialmente diverso
- Ogni istruzione viene eseguita un certo # di volte, dipendente da n

```
int min(int[] A, int n)
```

```
int min = A[0]
```

```
for i = 1 to n - 1 do
```

```
    if A[i] < min then
        min = A[i]
```

```
return min
```

Costo	# Volte
c_1	1
c_2	n
c_3	$n - 1$
c_4	$n - 1$
c_5	1

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_4(n - 1) + c_5$$

$$= (c_2 + c_3 + c_4)n + (c_1 + c_5 - c_3 - c_4) = an + b$$

Tempo di calcolo `binarySearch()`

Domanda

Come si valuta il tempo di calcolo di una funzione ricorsiva?

```
int binarySearch(int[] A, int v, int start, int end)
if start > end then
    return -1                                % Vettore vuoto
else
    int m =  $\lfloor (start + end) / 2 \rfloor$ 
    if A[m] == v then
        return m                            % Trovato
    else if A[m] < v then
        return binarySearch(A, v, m + 1, end) % Sottovettore di destra
    else
        return binarySearch(A, v, start, m - 1) % Sottovettore di sinistra
```

Algoritmi ricorsivi e ricorrenze

Relazione di ricorrenza

Quando un algoritmo richiama se stesso su un input più piccolo, il suo costo $T(n)$ dipende dal costo della chiamata ricorsiva.

Schema tipico

$$T(n) = \begin{cases} \text{costo del caso base} & n \leq n_0, \\ \text{costo delle chiamate ricorsive} + \text{lavoro locale} & n > n_0. \end{cases}$$

La ricerca binaria fornisce il primo esempio concreto; più avanti vedremo metodi sistematici per analizzare queste relazioni.

Tempo di calcolo `binarySearch()`

Il vettore viene suddiviso in due parti:

Parte SX: $\lfloor (n-1)/2 \rfloor$

Parte DX: $\lfloor n/2 \rfloor$

```
int binarySearch(int[] A, int v, int i, int j)
```

	Costo	# ($i > j$)	# ($i \leq j$)
if $i > j$ then	c_1	1	1
return -1	c_2	1	0
else			
int $m = \lfloor (i+j)/2 \rfloor$	c_3	0	1
if $A[m] == v$ then	c_4	0	1
return m	c_5	0	0
else if $A[m] < v$ then	c_6	0	1
return <code>binarySearch</code> ($A, v, m+1, j$)	$c_7 + T(\lfloor n/2 \rfloor)$	0	0/1
else			
return <code>binarySearch</code> ($A, v, i, m-1$)	$c_7 + T(\lfloor (n-1)/2 \rfloor)$	0	1/0

Tempo di calcolo `binarySearch()`

- **Assunzioni** (caso peggiore):

- Per semplicità, assumiamo n potenza di 2: $n = 2^k$
- L'elemento cercato non è presente
- Ad ogni passo, scegliamo sempre la parte DX di dimensione $n/2$

- **Due casi:**

$$i > j \quad (n = 0) \qquad T(n) = c_1 + c_2 = c$$

$$\begin{aligned} i \leq j \quad (n > 0) \qquad T(n) &= T(n/2) + c_1 + c_3 + c_4 + c_6 + c_7 \\ &= T(n/2) + d \end{aligned}$$

- **Relazione di ricorrenza:**

$$T(n) = \begin{cases} c & n = 0 \\ T(n/2) + d & n \geq 1 \end{cases}$$

Tempo di calcolo `binarySearch()`

Soluzione della relazione di ricorrenza tramite espansione

$$\begin{aligned}T(n) &= T(n/2) + d \\&= T(n/4) + 2d \\&= T(n/8) + 3d\end{aligned}$$

...

$$\begin{aligned}&= T(1) + kd \\&= T(0) + (k + 1)d \\&= kd + (c + d) \\&= d \log n + e.\end{aligned}$$

$$n = 2^k \Rightarrow k = \log n$$

Ordini di complessità

Per ora, abbiamo analizzato precisamente due algoritmi e abbiamo ottenuto due *funzioni di complessità*:

- Ricerca: $T(n) = d \log n + e$ **logaritmica**
- Minimo: $T(n) = an + b$ **lineare**

Una terza funzione deriva dall'*algoritmo naïf* per il minimo:

- Minimo: $T(n) = fn^2 + gn + h$ **quadratica**

Classi di complessità

$f(n)$	$n = 10$	$n = 10^2$	$n = 10^3$	$n = 10^4$	Tipo
$\log_2 n$	$3,3 \cdot 10^0$	$6,6 \cdot 10^0$	$1,0 \cdot 10^1$	$1,3 \cdot 10^1$	logaritmica
n	$1,0 \cdot 10^1$	$1,0 \cdot 10^2$	$1,0 \cdot 10^3$	$1,0 \cdot 10^4$	lineare
$n \log_2 n$	$3,3 \cdot 10^1$	$6,6 \cdot 10^2$	$1,0 \cdot 10^4$	$1,3 \cdot 10^5$	loglineare
n^2	$1,0 \cdot 10^2$	$1,0 \cdot 10^4$	$1,0 \cdot 10^6$	$1,0 \cdot 10^8$	quadratica
n^3	$1,0 \cdot 10^3$	$1,0 \cdot 10^6$	$1,0 \cdot 10^9$	$1,0 \cdot 10^{12}$	cubica
2^n	$1,0 \cdot 10^3$	$1,3 \cdot 10^{30}$	$1,1 \cdot 10^{301}$	$2,0 \cdot 10^{3010}$	esponenziale
$n!$	$3,6 \cdot 10^6$	$9,3 \cdot 10^{157}$	$4,0 \cdot 10^{2567}$	$2,8 \cdot 10^{35659}$	fattoriale

Come sbagliare completamente l'algoritmo di controllo degli update in Windows XP e renderlo esponenziale:

<http://m.slashdot.org/story/195683>

Algoritmi e Strutture Dati

Analisi di algoritmi Funzioni di costo, notazione asintotica

Alberto Montresor

Università di Trento

2026/08/21

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Notazioni O , Ω , Θ

Definizione – Notazione O

Sia $g(n)$ una funzione di costo; indichiamo con $\mathcal{O}(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\exists c > 0, \exists m \geq 0 : f(n) \leq cg(n), \forall n \geq m$$

- Come si legge: $f(n)$ è “**O grande**” (big-O) di $g(n)$
- Scrittura formale: $f(n) \in \mathcal{O}(g(n))$
- Scrittura convenzionale: $f(n) = \mathcal{O}(g(n))$
- $g(n)$ è un **limite asintotico superiore** per $f(n)$
- $f(n)$ cresce al più come $g(n)$

Notazioni O , Ω , Θ

Definizione – Notazione Ω

Sia $g(n)$ una funzione di costo; indichiamo con $\Omega(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\exists c > 0, \exists m \geq 0 : f(n) \geq cg(n), \forall n \geq m$$

- Come si legge: $f(n)$ è “**Omega grande**” di $g(n)$
- Scrittura formale: $f(n) \in \Omega(g(n))$
- Scrittura convenzionale: $f(n) = \Omega(g(n))$
- $g(n)$ è un **limite asintotico inferiore** per $f(n)$
- $f(n)$ cresce almeno quanto $g(n)$

Notazioni O , Ω , Θ

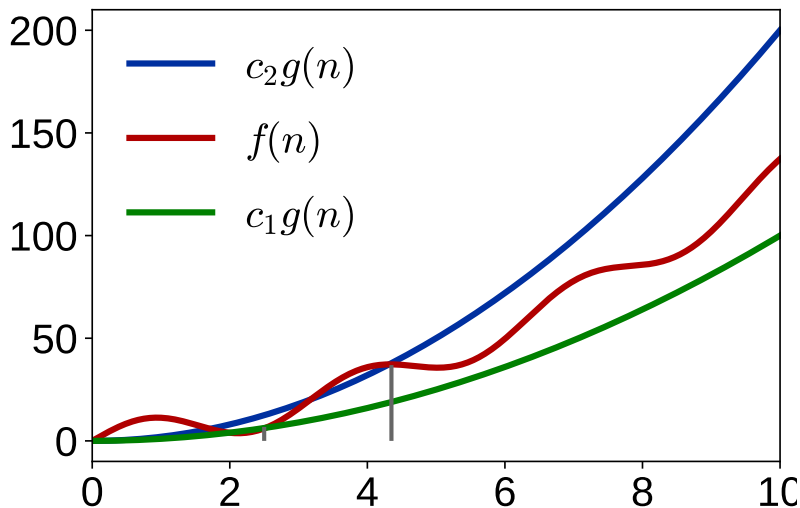
Definizione – Notazione Θ

Sia $g(n)$ una funzione di costo; indichiamo con $\Theta(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\exists c_1 > 0, \exists c_2 > 0, \exists m \geq 0 : c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq m$$

- Come si legge: $f(n)$ è “**Theta**” di $g(n)$
- Scrittura formale: $f(n) \in \Theta(g(n))$
- Scrittura convenzionale: $f(n) = \Theta(g(n))$
- $f(n)$ cresce esattamente come $g(n)$
- $f(n) = \Theta(g(n))$ se e solo se $f(n) = \mathcal{O}(g(n))$ e $f(n) = \Omega(g(n))$

Graficamente



Vero o falso?

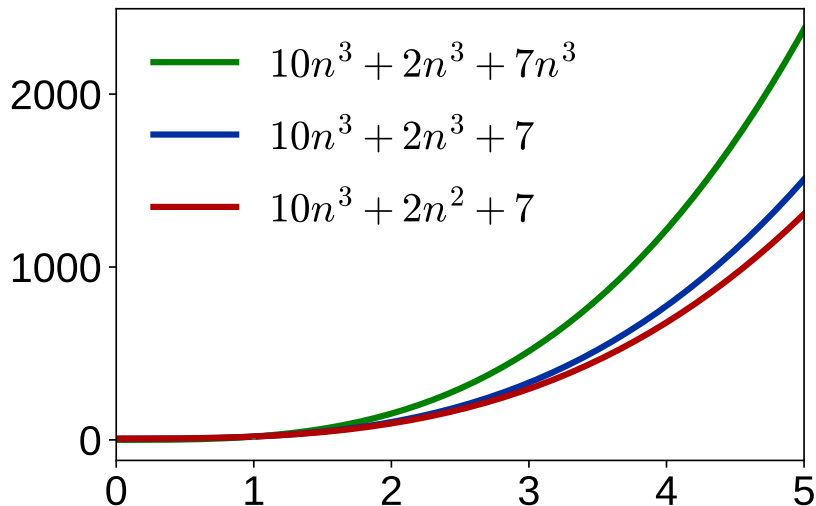
$$f(n) = 10n^3 + 2n^2 + 7 \stackrel{?}{=} \mathcal{O}(n^3)$$

Dobbiamo provare che $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^3, \forall n \geq m$

$$\begin{aligned} f(n) &= 10n^3 + 2n^2 + 7 \\ &\leq 10n^3 + 2n^3 + 7 && \forall n \geq 1 \\ &\leq 10n^3 + 2n^3 + 7n^3 && \forall n \geq 1 \\ &= 19n^3 \\ &\stackrel{?}{\leq} cn^3 \end{aligned}$$

che è vera per ogni $c \geq 19$ e per ogni $n \geq 1$, quindi $m = 1$.

Graficamente



Non è l'unico modo di procedere

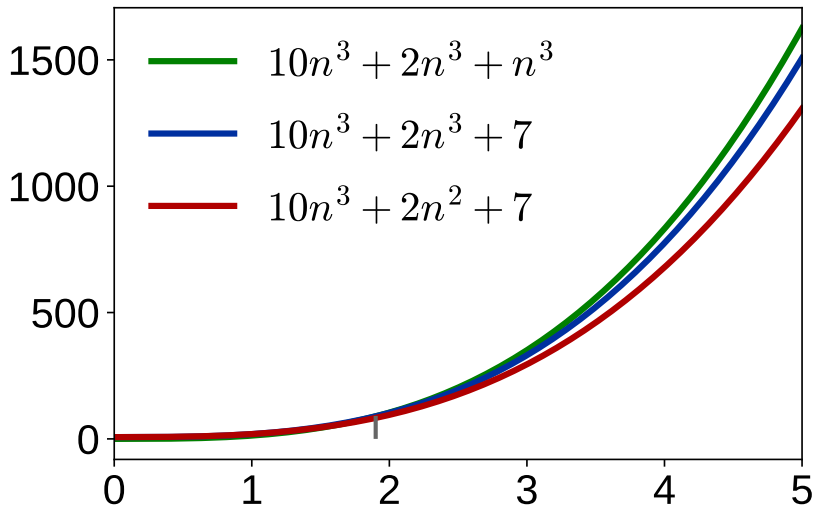
$$f(n) = 10n^3 + 2n^2 + 7 \stackrel{?}{=} \mathcal{O}(n^3)$$

Dobbiamo provare che $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^3, \forall n \geq m$

$$\begin{aligned} f(n) &= 10n^3 + 2n^2 + 7 \\ &\leq 10n^3 + 2n^3 + 7 && \forall n \geq 1 \\ &\leq 10n^3 + 2n^3 + n^3 && \forall n \geq \sqrt[3]{7} \\ &= 13n^3 \\ &\stackrel{?}{\leq} cn^3 \end{aligned}$$

che è vera per ogni $c \geq 13$ e per ogni $n \geq \sqrt[3]{7}$, quindi usiamo $m = 2$

Graficamente



Vero o falso?

$$f(n) = 3n^2 + 7n \stackrel{?}{=} \Theta(n^2)$$

Limite inferiore: $\exists c_1 > 0, \exists m_1 \geq 0 : f(n) \geq c_1 n^2, \forall n \geq m_1$

$$f(n) = 3n^2 + 7n$$

$$\geq 3n^2$$

Per $n \geq 0$

$$\stackrel{?}{\geq} c_1 n^2$$

che è vera per ogni $c_1 \leq 3$ e per ogni $n \geq 0$, quindi $m_1 = 0$

Vero o falso?

$$f(n) = 3n^2 + 7n \stackrel{?}{=} \Theta(n^2)$$

Limite superiore: $\exists c_2 > 0, \exists m_2 \geq 0 : f(n) \leq c_2 n^2, \forall n \geq m_2$

$$\begin{aligned} f(n) &= 3n^2 + 7n \\ &\leq 3n^2 + 7n^2 && \text{Per } n \geq 1 \\ &= 10n^2 \\ &\stackrel{?}{\leq} c_2 n^2 \end{aligned}$$

che è vera per ogni $c_2 \geq 10$ e per ogni $n \geq 1$, quindi $m_2 = 1$

Vero o falso?

$$f(n) = 3n^2 + 7n \stackrel{?}{=} \Theta(n^2)$$

Notazione Θ :

$$\exists c_1 > 0, \exists c_2 > 0, \exists m \geq 0 : c_1 n^2 \leq f(n) \leq c_2 n^2, \forall n \geq m$$

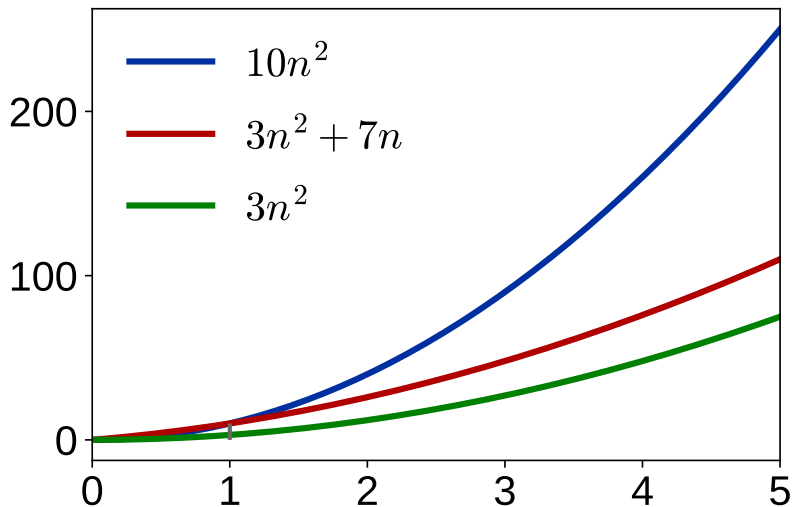
Con questi parametri:

$$c_1 = 3$$

$$c_2 = 10$$

$$m = \max\{m_1, m_2\} = \max\{0, 1\} = 1$$

Graficamente



Vero o falso?

$$n^2 \stackrel{?}{\in} \mathcal{O}(n)$$

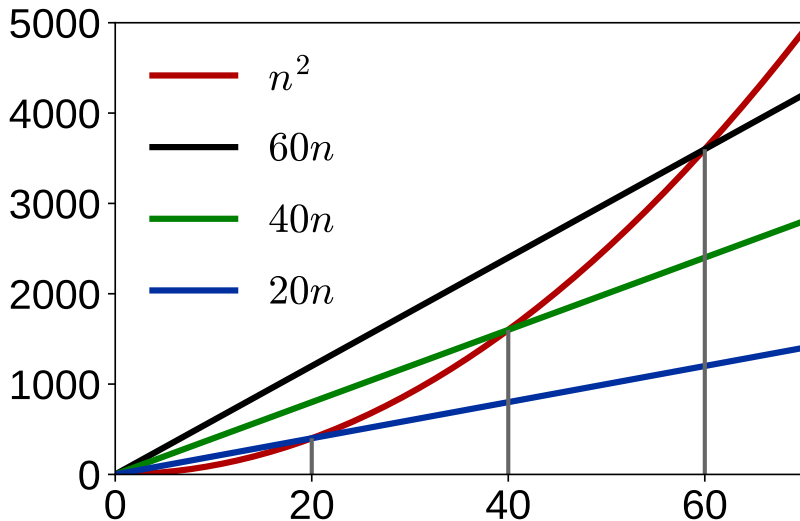
Dobbiamo dimostrare che $\exists c > 0, \exists m \geq 0 : n^2 \leq cn, \forall n \geq m$

- Dovrebbe esistere una costante $c > 0$ tale che

$$n^2 \leq cn \iff c \geq n.$$

- Ma n cresce senza limite: nessuna costante c può soddisfare la disequazione per ogni $n \geq m$.

Graficamente



Vero o falso?

$$n^2 \stackrel{?}{=} \mathcal{O}(n^3)$$

Dobbiamo dimostrare che $\exists c > 0, \exists m \geq 0 : n^2 \leq cn^3, \forall n \geq m$

- Otteniamo questo: $n^2 \leq cn^3 \Leftrightarrow c \geq \frac{1}{n}$
- La funzione $1/n$ è monotona decrescente per $n > 0$.
- Possiamo scegliere $m = 1$ e $c = 1$: per ogni $n \geq 1$ vale infatti $1/n \leq 1$.

Algoritmi e Strutture Dati

Analisi di algoritmi Proprietà della notazione asintotica

Alberto Montresor

Università di Trento

2026/08/21

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Proprietà

Dualità

$$f(n) = \mathcal{O}(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$$

Dimostrazione:

$$f(n) = \mathcal{O}(g(n)) \Leftrightarrow f(n) \leq cg(n), \forall n \geq m$$

$$\Leftrightarrow g(n) \geq \frac{1}{c} f(n), \forall n \geq m$$

$$\Leftrightarrow g(n) \geq c' f(n), \forall n \geq m$$

$$\Leftrightarrow g(n) = \Omega(f(n))$$

$$c' = \frac{1}{c}$$

Proprietà

Eliminazione delle costanti

$$f(n) = \mathcal{O}(g(n)) \Leftrightarrow af(n) = \mathcal{O}(g(n)), \forall a > 0$$

$$f(n) = \Omega(g(n)) \Leftrightarrow af(n) = \Omega(g(n)), \forall a > 0$$

Dimostrazione:

$$f(n) = \mathcal{O}(g(n)) \Leftrightarrow f(n) \leq cg(n), \forall n \geq m$$

$$\Leftrightarrow af(n) \leq acg(n), \forall n \geq m, \forall a > 0$$

$$\Leftrightarrow af(n) \leq c'g(n), \forall n \geq m$$

$$c' = ac > 0$$

$$\Leftrightarrow af(n) = \mathcal{O}(g(n))$$

Proprietà

Sommatoria (Sequenza di algoritmi)

$$f_i(n) = \mathcal{O}(g_i(n)), i = 1, 2 \Rightarrow f_1(n) + f_2(n) = \mathcal{O}(\max\{g_1(n), g_2(n)\})$$

$$f_i(n) = \Omega(g_i(n)), i = 1, 2 \Rightarrow f_1(n) + f_2(n) = \Omega(\max\{g_1(n), g_2(n)\})$$

Dimostrazione (Lato $\mathcal{O}$)

$$\begin{aligned} f_1(n) + f_2(n) &\leq c_1 g_1(n) + c_2 g_2(n) \\ &\leq (c_1 + c_2) \max\{g_1(n), g_2(n)\} \\ &\Rightarrow f_1(n) + f_2(n) = \mathcal{O}(\max\{g_1(n), g_2(n)\}). \end{aligned}$$

Proprietà

Prodotto (Cicli annidati)

$$f_1(n) = \mathcal{O}(g_1(n)), f_2(n) = \mathcal{O}(g_2(n)) \Rightarrow f_1(n) \cdot f_2(n) = \mathcal{O}(g_1(n) \cdot g_2(n))$$

$$f_1(n) = \Omega(g_1(n)), f_2(n) = \Omega(g_2(n)) \Rightarrow f_1(n) \cdot f_2(n) = \Omega(g_1(n) \cdot g_2(n))$$

Dimostrazione

$$f_1(n) = \mathcal{O}(g_1(n)) \wedge f_2(n) = \mathcal{O}(g_2(n)) \Rightarrow$$

$$f_1(n) \leq c_1 g_1(n) \wedge f_2(n) \leq c_2 g_2(n) \Rightarrow$$

$$f_1(n) \cdot f_2(n) \leq c_1 c_2 g_1(n) g_2(n)$$

Proprietà

Simmetria

$$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$$

Dimostrazione

Grazie alla proprietà di dualità:

$$f(n) = \Theta(g(n)) \Rightarrow f(n) = \mathcal{O}(g(n)) \Rightarrow g(n) = \Omega(f(n))$$

$$f(n) = \Theta(g(n)) \Rightarrow f(n) = \Omega(g(n)) \Rightarrow g(n) = \mathcal{O}(f(n))$$

L'implicazione inversa si ottiene scambiando f e g .

Proprietà

Transitività

$$f(n) = \mathcal{O}(g(n)), g(n) = \mathcal{O}(h(n)) \Rightarrow f(n) = \mathcal{O}(h(n))$$

Dimostrazione

$$f(n) = \mathcal{O}(g(n)) \wedge g(n) = \mathcal{O}(h(n)) \Rightarrow$$

$$f(n) \leq c_1 g(n) \wedge g(n) \leq c_2 h(n) \Rightarrow$$

$$f(n) \leq c_1 c_2 h(n) \Rightarrow$$

$$f(n) = \mathcal{O}(h(n))$$

Espressioni polinomiali

Espressioni polinomiali

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0, a_k > 0 \Rightarrow f(n) = \Theta(n^k)$$

Limite superiore: $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^k, \forall n \geq m$

$$\begin{aligned} f(n) &= a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \\ &\leq a_k n^k + |a_{k-1}| n^{k-1} + \dots + |a_1| n + |a_0| \\ &\leq a_k n^k + |a_{k-1}| n^k + \dots + |a_1| n^k + |a_0| n^k & \forall n \geq 1 \\ &= (a_k + |a_{k-1}| + \dots + |a_1| + |a_0|) n^k \\ &\stackrel{?}{\leq} cn^k \end{aligned}$$

che è vera per $c \geq (a_k + |a_{k-1}| + \dots + |a_1| + |a_0|) > 0$ e per $m = 1$.

Espressioni polinomiali

Espressioni polinomiali

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0, a_k > 0 \Rightarrow f(n) = \Theta(n^k)$$

Limite inferiore: $\exists d > 0, \exists m \geq 0 : f(n) \geq dn^k, \forall n \geq m$

$$\begin{aligned} f(n) &= a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \\ &\geq a_k n^k - |a_{k-1}| n^{k-1} - \dots - |a_1| n - |a_0| \\ &\geq a_k n^k - |a_{k-1}| n^{k-1} - \dots - |a_1| n^{k-1} - |a_0| n^{k-1} \quad \forall n \geq 1 \\ &\stackrel{?}{\geq} dn^k \end{aligned}$$

L'ultima equazione è vera se:

$$d \leq a_k - \frac{|a_{k-1}|}{n} - \frac{|a_{k-2}|}{n} - \dots - \frac{|a_1|}{n} - \frac{|a_0|}{n} > 0 \Leftrightarrow n > \frac{|a_{k-1}| + \dots + |a_0|}{a_k}$$

Costo costante

- Qual è la complessità di $f(n) = 17$?

- $f(n) = 17 \geq c_1 n^0 \Rightarrow c_1 \leq 17$
- $f(n) = 17 \leq c_2 n^0 \Rightarrow c_2 \geq 17$
- $f(n) = \Theta(n^0) = \Theta(1)$

Iterazioni limitate da un valore costante (non dipendente da n) hanno costo costante

```
real distance3D(real[] p, real[] q)
```

```
real s = 0
```

```
for i = 0 to 2 do
```

```
     $s = s + (p[i] - q[i])^2$ 
```

```
return  $\sqrt{s}$ 
```

Logaritmi vs funzioni polinomiali

Proprietà dei logaritmi

Per ogni costante $k > 0$, $\log n = \mathcal{O}(n^k)$.

Per la concavità del logaritmo, $\ln x \leq x - 1 \quad \forall x > 0$.

Ponendo $x = n^k$, con $k > 0$, otteniamo

$$\ln(n^k) \leq n^k - 1$$

$$k \ln n \leq n^k - 1$$

$$\ln n \leq \frac{n^k - 1}{k} \leq \frac{1}{k} n^k$$

$$\log n \leq \frac{1}{k \ln 2} n^k$$

$$\ln n = \ln 2 \log n$$

Quindi, scegliendo $c = \frac{1}{k \ln 2}$, si ha $\log n = \mathcal{O}(n^k)$ per ogni $k > 0$.

Prendiamo confidenza

- È vero che $\log_a n = \Theta(\log n)$?
 - Sì: $\log_a n = (\log_a 2) \cdot (\log_2 n) = \Theta(\log n)$
- È vero che $\log n^a = \Theta(\log n)$, per $a > 0$?
 - Sì: $\log n^a = a \log n = \Theta(\log n)$
- È vero che $2^{n+1} = \Theta(2^n)$?
 - Sì: $2^{n+1} = 2 \cdot 2^n = \Theta(2^n)$
- È vero che $2^n = \Theta(3^n)$?
 - Ovviamente $2^n = \mathcal{O}(3^n)$
 - Ma: $3^n = \left(\frac{3}{2} \cdot 2\right)^n = \left(\frac{3}{2}\right)^n \cdot 2^n$:
Quindi non esiste $c > 0$ tale per cui $\left(\frac{3}{2}\right)^n \cdot 2^n \leq c 2^n$, e quindi
 $2^n \neq \Omega(3^n)$

Classificazione delle funzioni

È possibile ottenere un ordinamento delle principali espressioni, estendendo le relazioni che abbiamo dimostrato fino ad ora

Per ogni $0 < r < s, 0 < h < k, 1 < a < b$:

$$\mathcal{O}(1) \subset \mathcal{O}(\log^r n) \subset \mathcal{O}(\log^s n) \subset \mathcal{O}(n^h) \subset \mathcal{O}(n^h \log^r n) \subset \mathcal{O}(n^h \log^s n) \subset \mathcal{O}(n^k) \subset \mathcal{O}(a^n) \subset \mathcal{O}(b^n) \subset \mathcal{O}(n!) \subset \mathcal{O}(n^n)$$

Notazioni o, ω

Definizione – Notazioni o, ω

Sia $g(n)$ una funzione di costo; indichiamo con $o(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\forall c > 0, \exists m \geq 0 : 0 \leq f(n) < cg(n), \forall n \geq m.$$

Sia $g(n)$ una funzione di costo; indichiamo con $\omega(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\forall c > 0, \exists m \geq 0 : f(n) > cg(n) \geq 0, \forall n \geq m.$$

- Come si leggono: $f(n)$ è “**o piccolo**”, “**omega piccolo**” di $g(n)$
- Come si scrivono: $f(n) = o(g(n))$ oppure $f(n) = \omega(g(n))$

Notazioni o, ω

Utilizzando il concetto di limite, date due funzioni $f(n)$ e $g(n)$ si possono fare le seguenti affermazioni:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f(n) = o(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c, \quad 0 < c < +\infty \Rightarrow f(n) = \Theta(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty \Rightarrow f(n) = \omega(g(n))$$

Si noti che:

$$f(n) = o(g(n)) \Rightarrow f(n) = \mathcal{O}(g(n))$$

$$f(n) = \omega(g(n)) \Rightarrow f(n) = \Omega(g(n))$$